

Reinforcement Learning

Assignment 1

Authors:

Martin Nilsson, *School of Engineering*, Jönköping University

Besan Ewir, *School of Engineering*, Jönköping University

Vladimir Giustacchini, *School of Engineering*, Jönköping University

Jorge Cuello, *School of Engineering*, Jönköping University

September 16, 2026

Contents

1	Assignment 1: Reinforcement Learning	2
1.1	Question 1	2
1.2	Question 2	4
1.1	Question 3	6
2	Practical Part	6

1 Theoretical Part

1.1 Question 1

Q1: “Prove that the policy iteration algorithm converges to the optimal policy for a finite MDP with a finite action space for each state.”

First of all, its good to define each key word:

Optimal Policy: An optimal policy is a strategy or decision-making rule that maximizes the expected cumulative reward for an agent in a given environment. An optimal policy is one that leads to the highest possible long-term reward for the agent, considering the dynamics of the environment and the available actions.

Finite MDP: A Markov Decision Process (MDP) is used to model the decision-making problems where an agent interacts with an environment over discrete time steps. The keyword "finite" only means that the MDP consists of a limited set of states and actions with respective transition function that defines the probabilities of moving from one state to another given an action, and a reward function that assigns a numerical value to each state-action pair.

Finite Action Space: A finite action space refers to a situation in which an agent has a limited number of possible actions to choose from in each state of the environment. Meaning that, for every state in the Markov Decision Process (MDP), there is a finite set of actions that the agent can take.

Policy iteration contains a three different steps:

1. **Initialization:** Start with an arbitrary policy π_0 and initialize the value function $V_0(s)$ for all states s .
2. **Policy Evaluation:** For the current policy π_k , the value function $V_k(s)$ is computed for all states s using the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

However, for a deterministic policy, the action is chosen with probability 1, so the equation simplifies to:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

3. **Policy Improvement:** The policy π_{k+1} is updated by choosing the action that maximizes the expected value for each state:

$$\pi'(s) = \arg \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

After a number of iterations, the policy will converge to the optimal policy: $\pi = \pi^*$, and the value function will converge to the optimal value function $V = V^*$.

Main idea: Policy Improvement cannot make the policy worse

Define the action-value function under policy π as:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

Since V^π is the value of following π ,

$$V^\pi(s) = Q^\pi(s, \pi(s))$$

The policy improvement step chooses:

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

Therefore,

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

So we have

$$R(s, \pi'(s)) + \gamma \sum_{s'} P(s'|s, \pi'(s)) V^\pi(s') \geq V^\pi(s) \quad (1)$$

This says that taking the improved action once, and then following the old policy, is at least as good as following the old policy immediately.

From (1), we apply the reasoning recursively to get:

$$V^\pi(s) \leq \mathbb{E}_{\pi'} [R_0 + \gamma R_1 + \gamma R_2 + \dots]$$

On the right side we simply get the expected value for following the new policy π' from s , which is $V^{\pi'}(s)$. So we have,

$$V^{\pi'}(s) \geq V^\pi(s), \quad \forall s \in \mathcal{S}$$

the algorithm will eventually converge to the optimal policy because of the finite number of states and actions $\Rightarrow$ (finite number of policies) so the policy improvement must eventually converge to the optimal policy.

Basically the idea of policy improvement is based of four main points:

- Policy improvement $\Rightarrow V^{\pi_{k+1}} \geq V^{\pi_k}$
- Finite number of policies $\Rightarrow$ algorithm must terminate
- Termination $\Rightarrow$ no action can improve the policy
- No improving action $\Rightarrow$ Bellman optimality equation $\Rightarrow \pi = \pi^*$

1.2 Question 2

Q2: “Prove that the Bellman optimality equation has a unique solution for a finite MDP with a bounded reward function.”

Setup

Consider a finite MDP with state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$, transition probabilities $P(s' | s, a)$, discount factor $\gamma \in [0, 1)$, and a *bounded* reward function: there exists $R_{\max} < \infty$ with $|R(s, a, s')| \leq R_{\max}$ for all s, a, s' .

Since $\mathcal{S}$ is finite, a value function is just a vector $V \in \mathbb{R}^{|\mathcal{S}|}$. We measure distance between two value functions with the **max-norm**:

$$\|V - U\|_{\infty} = \max_{s \in \mathcal{S}} |V(s) - U(s)|.$$

This space is *complete* (finite-dimensional real vector spaces always are).

Define the **Bellman optimality operator** T , which takes a value function V and produces a new one:

$$(TV)(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')].$$

A solution to the Bellman optimality equation is exactly a **fixed point** of T , i.e. a V^* with $TV^* = V^*$.

Step 1: T is a contraction

Theorem 1.1 For any two value functions V, U ,

$$\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty.$$

Proof. Fix a state s . Both $(TV)(s)$ and $(TU)(s)$ are maxima over the same finite action set, so their difference is bounded by the largest per-action gap:

$$|(TV)(s) - (TU)(s)| \leq \max_a \left| \sum_{s'} P(s' | s, a) \gamma (V(s') - U(s')) \right|.$$

(Intuitively: whichever action attains the max for V can be no more than as far “ahead” of U ’s best action as the two functions ever differ, and vice versa the reward terms are identical for V and U and cancel out.)

Since $P(\cdot | s, a)$ is a probability distribution, this average is at most γ times the worst-case disagreement between V and U :

$$\left| \sum_{s'} P(s' | s, a) \gamma (V(s') - U(s')) \right| \leq \gamma \sum_{s'} P(s' | s, a) |V(s') - U(s')| \leq \gamma \|V - U\|_\infty.$$

This bound holds for every state s , so taking the max over s gives

$$\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty. \quad \square$$

Since $\gamma < 1$, this says T strictly shrinks distances exactly like the photocopier at $\gamma \times 100\%$ zoom.

Step 2: Contractions have a unique fixed point

Theorem 1.2 – Banach Fixed-Point Theorem If T is a contraction on a complete space (i.e. $\|TV - TU\| \leq \gamma \|V - U\|$ for some $\gamma < 1$), then T has **exactly one** fixed point, and repeatedly applying T from any starting point converges to it.

We’ve shown T satisfies exactly this hypothesis. So this theorem hands us existence, uniqueness, and convergence all at once, with no extra work.

A direct one-line uniqueness check (optional, self-contained)

Suppose V_1 and V_2 are both solutions: $TV_1 = V_1$ and $TV_2 = V_2$. Then by Step 1,

$$\|V_1 - V_2\|_\infty = \|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty.$$

Since $\gamma < 1$, the only number $d \geq 0$ satisfying $d \leq \gamma d$ is $d = 0$. So $\|V_1 - V_2\|_\infty = 0$, meaning $V_1 = V_2$.

Conclusion

Theorem 1.1 For a finite MDP with bounded rewards and $\gamma \in [0, 1)$, the Bellman optimality equation has a unique solution V^* .

Proof. T is a contraction (Step 1) on a complete space, so by the Banach Fixed-Point Theorem (Step 2) it has exactly one fixed point i.e. the Bellman optimality equation has exactly one solution. $\square$

Bonus: this same argument is why *value iteration* works. Value iteration is literally $V_{k+1} = TV_k$ repeatedly photocopying. Banach's theorem guarantees this converges to V^* no matter what V_0 you start from.

This answer is partially inspired by the explanation in [1].

1.1 Question 3

Q3: “Analyze the challenges of using linear function approximation in value-based reinforcement learning. Discuss the potential issues that may arise and propose methods to mitigate them”

In reinforcement learning, tabular methods store a separate value for every state or state-action pair. This works well for small problems, but it becomes impractical when the state space is very large or continuous. Function approximation addresses this shortcoming by using fewer parameters to describe the value function. In linear function approximation, the state-value function is defined by

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s),$$

where $\mathbf{x}(s)$ is the feature vector that represents the state s and $\mathbf{w}$ is the vector of learned weights. A similar representation can be used for action values:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a).$$

Linear function approximation is computationally efficient and allows learning to generalize between similar states, but it also brings a new set of challenges.

1. Approximation Error

Linear function approximation can only represent value functions that can be expressed using the selected features.

Suppose

$$\hat{v}(s, \mathbf{w}) = w_1 x_1 + w_2 x_2$$

but the true value is

$$v_\pi(s) = 2x_1 + 3x_2 + 4x_1x_2.$$

The term $4x_1x_2$ cannot be represented using only x_1 and x_2 . Therefore, some approximation error may remain even after training.

A possible mitigation is to introduce richer features. For example, an **interaction feature** can be added:

$$x_3 = x_1x_2.$$

The approximation then becomes

$$\hat{v}(s, \mathbf{w}) = w_1 x_1 + w_2 x_2 + w_3 x_1 x_2.$$

If the true value function is

$$v_\pi(s) = 2x_1 + 3x_2 + 4x_1x_2,$$

choosing

$$(w_1, w_2, w_3) = (2, 3, 4)$$

represents it exactly. The model is still linear in the weights, but the addition of the interaction term x_1x_2 adds to the expressiveness of the model. Other feature construction methods, such as tile coding, can also provide richer representations and reduce approximation error.

2. Feature Selection

The approximation depends directly on the feature vector:

$$\mathbf{x}(s) = [x_1(s), x_2(s), \dots, x_n(s)]^T.$$

If important information is missing from $\mathbf{x}(s)$, the weights cannot recover it.

For example, suppose two states in the same gridworld have the agent at the same position but different obstacle configurations. If the features contain only the agent position:

$$\mathbf{x}(s_1) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \mathbf{x}(s_2) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

Then

$$\hat{v}(s_1) = \mathbf{w}^T \mathbf{x}(s_1) = \mathbf{w}^T \mathbf{x}(s_2) = \hat{v}(s_2).$$

The model is **forced to give them the same value**, even if

$$v_\pi(s_1) \neq v_\pi(s_2).$$

A possible mitigation is feature engineering, where relevant information is added to the feature representation. For example,

$$\mathbf{x}(s) = [\text{agent position}, \text{obstacle information}]^T.$$

This allows the approximation to distinguish between states that would otherwise have identical features but different values.

3. Generalization and Interference

With linear function approximation, different states may share the same features and weights. Therefore, learning from one state can also change the estimated value of other states.

For example, suppose two states have the feature vectors

$$\mathbf{x}(s_1) = [1, 1, 0]$$

and

$$\mathbf{x}(s_2) = [1, 0, 1].$$

Both states share the first feature. Since the value estimate is

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s),$$

both states use the same weight w_1 . Therefore, whenever w_1 is updated during learning in state s_1 , there will be an effect on the estimation of the value of s_2 as well.

Such an effect is known as generalization, and it can be useful when the states are similar, since learning from one state can improve the value estimate of another. However, if the states are unrelated, the same update may incorrectly change the value of the other state. This unwanted effect is called **interference**.

A possible mitigation is to use more localized features, such as **tile coding**, so that similar states share features while unrelated states have less feature overlap.

4. Correlated Experience

In RL, training samples are sequential:

$$S_t \rightarrow S_{t+1} \rightarrow S_{t+2} \rightarrow \dots$$

so consecutive samples are often correlated rather than independent.

For example, an agent moving through a grid might observe

$$(2, 2) \rightarrow (3, 2) \rightarrow (4, 2) \rightarrow (5, 2).$$

These states are closely related. If the agent repeatedly follows similar trajectories, some parts of the state space receive many updates while others receive very few:

$$N(s_1) \gg N(s_2),$$

where $N(s)$ is the **number of visits to state** s .

The approximation may consequently become accurate for frequently visited states but poor for rarely visited states.

A mitigation is sufficient **exploration**, so that the agent investigate the environment to guarantee more effective coverage of the relevant state space. Another mitigation is **Experience Replay**, where diverse past experiences are stored and reused during training. This can reduce the strong temporal correlation between consecutive samples. However, if the policy changes, old experiences may become off-policy.

5. Non-Stationary and Bootstrapped Targets

TD learning uses a **bootstrapped target**

$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t).$$

The target is **bootstrapped** because it uses the current estimated value of the next state, $\hat{v}(S_{t+1}, \mathbf{w}_t)$, instead of waiting for the complete return.

However, this estimate depends on the learned weights $\mathbf{w}_t$. For example, suppose

$$\hat{v}(S_{t+1}, \mathbf{w}_t) = 5.$$

Then

$$U_t = 1 + 0.9(5) = 5.5.$$

After learning, the weights change. Suppose the new estimate becomes

$$\hat{v}(S_{t+1}, \mathbf{w}_{t+1}) = 6.$$

The corresponding target becomes

$$1 + 0.9(6) = 6.4.$$

This is the **non-stationary target problem**: while the model is learning toward the target, changes in the weights can also change the target itself.

One possible mitigation is to reduce or avoid bootstrapping. **Monte Carlo** instead uses the observed return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

without using $\hat{v}(S_{t+1})$ as the target.

Multi-step TD and TD(λ) provide a compromise:

$$\text{1-step TD} \longleftrightarrow \text{multi-step} \longleftrightarrow \text{Monte Carlo}.$$

This can reduce dependence on the current approximation, although Monte Carlo generally introduces higher variance.

6. Instability, Divergence, and the Deadly Triad

One of the most serious challenges occurs when three components are combined:

$$\boxed{\text{Function Approximation} + \text{Bootstrapping} + \text{Off-policy Learning}}$$

This combination is called the **Deadly Triad** and can cause the value estimates to diverge rather than converge.

Consider Q-learning with linear function approximation:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a)$$

and the TD error

$$\delta_t = R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}).$$

The three components can be seen directly in this update:

$$\underbrace{\hat{q}(s, a, \mathbf{w})}_{\text{function approximation}}, \quad \underbrace{R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w})}_{\text{bootstrapped target}}.$$

and Q-learning is **off-policy** because the action used in the target,

$$\arg \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}),$$

can be different from the action actually selected by the behaviour policy.

For example, suppose

$$R = 0, \quad \gamma = 0.9,$$

the current estimate is

$$\hat{q}(S_t, A_t) = 5,$$

and the estimated value of the best next action is

$$\max_{a'} \hat{q}(S_{t+1}, a') = 10.$$

The TD target is therefore

$$U_t = 0 + 0.9(10) = 9,$$

and the TD error becomes

$$\delta_t = 9 - 5 = 4.$$

Since the TD error is positive, the algorithm updates the weights to increase the current estimate $\hat{q}(S_t, A_t)$.

In terms of **linear function approximation**, the weight vector is shared by several state-action pairs. Thus, the weight update of the current estimate may also change other estimates, including the next-state estimate used for computing the bootstrapped target.

This feedback can make learning unstable. Under certain conditions, the estimates may keep on growing instead of reaching a steady value; such a situation is called divergence.

To mitigate this issue we can remove or weaken one component of the Deadly Triad. For example, an **on-policy method like SARSA** removes the off-policy component, and multi-step returns reduce the bootstrapping property. Reducing one of the elements can help reduce the risk of instability and divergence.

2 Practical part

2.1 Grid World MDP

The environment is modelled as a finite Markov Decision Process (MDP). The grid has 5 rows and 5 columns, giving a maximum of 25 cells. However, five cells are obstacles and cannot be occupied by the agent.

The state space $\mathcal{S}$ contains every non-obstacle grid cell:

$$\mathcal{S} = \{(r, c) \mid 0 \leq r < 5, 0 \leq c < 5, (r, c) \notin \text{Obstacles}\}$$

The agent starts in state $(4, 0)$, while the goal state is $(0, 4)$. The obstacle states are:

$$\text{Obstacles} = \{(1, 1), (2, 3), (3, 1), (3, 2), (4, 4)\}$$

The available actions are:

$$\mathcal{A} = \{\text{up, down, left, right}\}$$

The transition model is stochastic. When the agent selects an action, it performs the intended movement with probability 0.7. Each of the other three directions is taken with probability 0.1:

$$P(\text{intended direction}) = 0.7$$

$$P(\text{each unintended direction}) = 0.1$$

Therefore, the probabilities for all possible directions add up to 1.0. For example, when the agent chooses **up**, it moves up with probability 0.7, and moves down, left, or right with probability 0.1 each.

If a movement would take the agent outside the grid or into an obstacle, the agent remains in its current state. However, the attempted movement still determines the reward.

The reward function is defined as follows:

$$R(s, a, s') = \begin{cases} +5, & \text{if the attempted movement reaches the goal,} \\ -5, & \text{if the attempted movement enters an obstacle,} \\ -1, & \text{otherwise.} \end{cases}$$

Thus, reaching the goal is encouraged, attempting to move into an obstacle is strongly penalized, and all other movements, including movements outside the grid, receive a small penalty of -1 .

2.2 Value Iteration

The value iteration algorithm calculates the best expected reward for every state in the grid.

First, the value of every state is initialized to zero:

$$V(s) = 0$$

The algorithm then repeatedly visits each state and evaluates every possible action. For each action, it examines all possible transitions and calculates the expected reward using the transition probability, immediate reward, and discounted value of the next state:

$$Q(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

The largest action value becomes the new value of the state:

$$V_{\text{new}}(s) = \max_{a \in \mathcal{A}} Q(s, a)$$

After all states have been updated, the algorithm compares the new values with the previous values. The largest difference is stored in Δ . The process continues until this difference is smaller than the threshold θ , which means that the values have converged.

Finally, the algorithm returns the converged value dictionary. These values are then used to extract the best action for each state.

2.3 Policy Extraction

After the value iteration algorithm converges, it returns a value function $V(s)$ containing the final value of every state.

To extract the optimal policy, the algorithm visits every state and evaluates each possible action. For every action, it examines all possible transitions and calculates its expected value using the transition probability, reward, and discounted value of the next state:

$$Q(s, a) = \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V(s')]$$

The action values are stored in a dictionary. The action with the highest expected value is then selected as the best action for that state:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$$

This process is repeated for every state in the grid. The final policy is a dictionary that maps each state to its best action:

$$\pi^* : \mathcal{S} \rightarrow \mathcal{A}$$

Therefore, the value function tells us how valuable each state is, while the optimal policy uses these values to decide which action the agent should take from each state.

References

- [1] P. Salunkhe, “Understanding bellman operators: The engine of dynamic programming.” <https://medium.com/@salunkheprathmesh0/understanding-bellman-operators-the-engine-of-dynamic-programming-96eb9cb06e93>, 2023. Accessed: 2026-09-15.